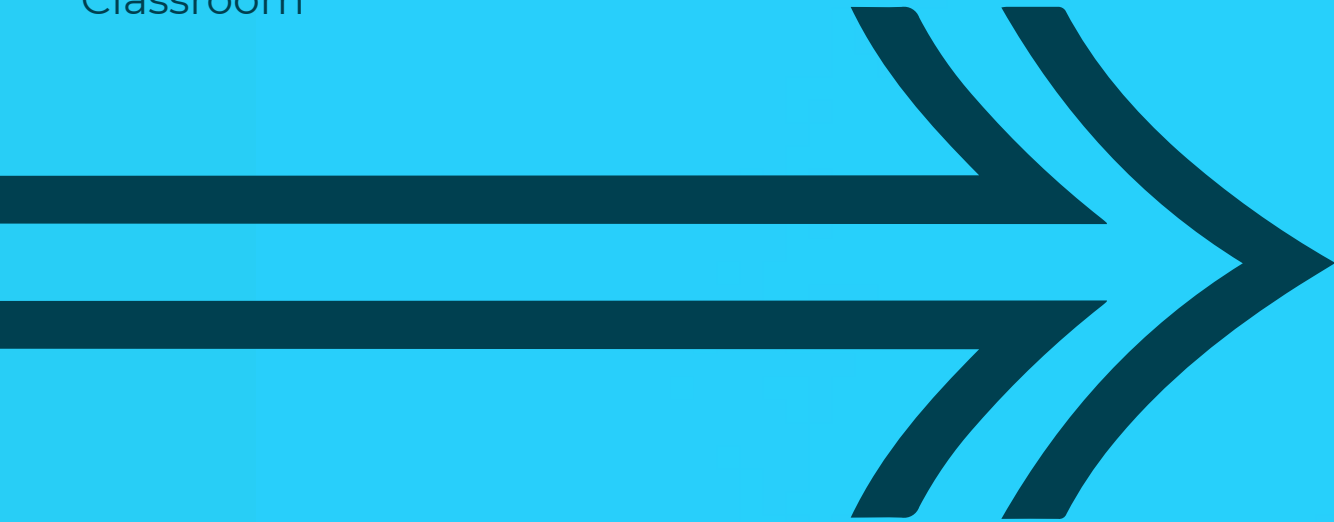




MICROSERVICES DESIGN

Classroom

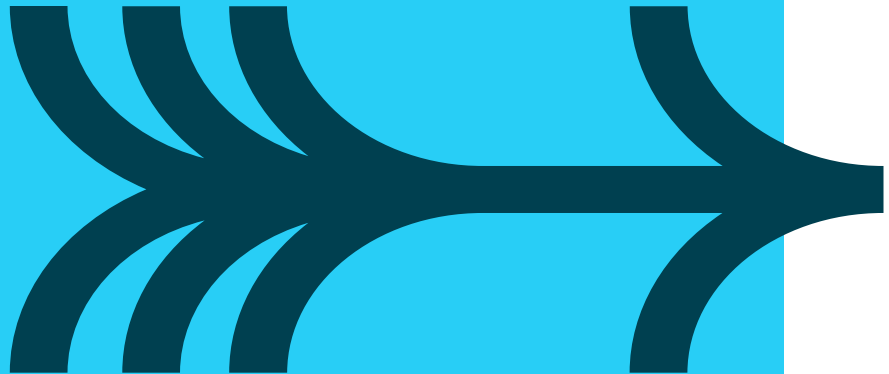




DOMAIN DRIVEN DESIGN & BOUNDED CONTEXT

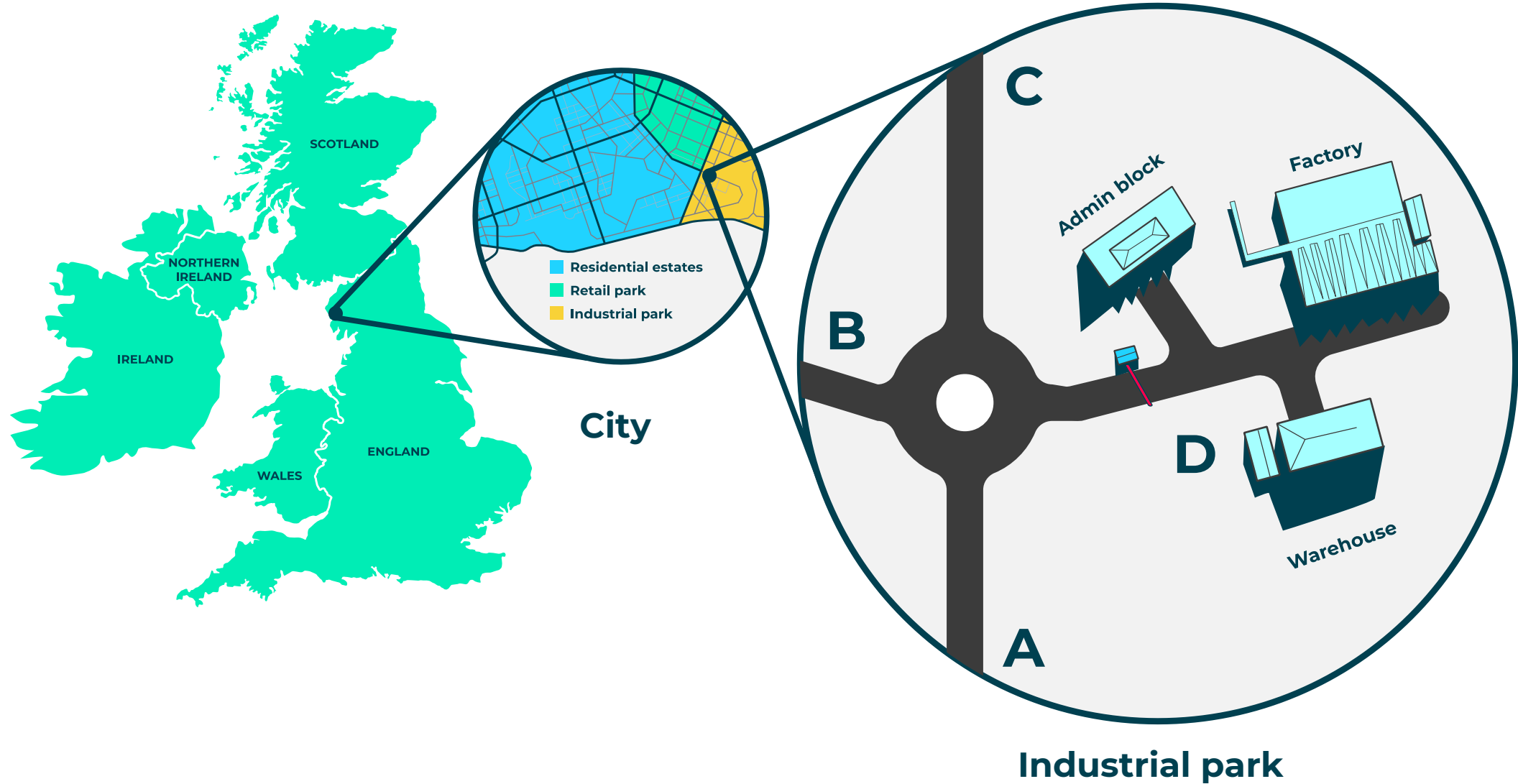
A Microservice has a well-defined boundary

- A **Domain Driven Design** aligns well with microservice design and has well defined integration points with other domains
- A **Bounded Context** encapsulates a Domain



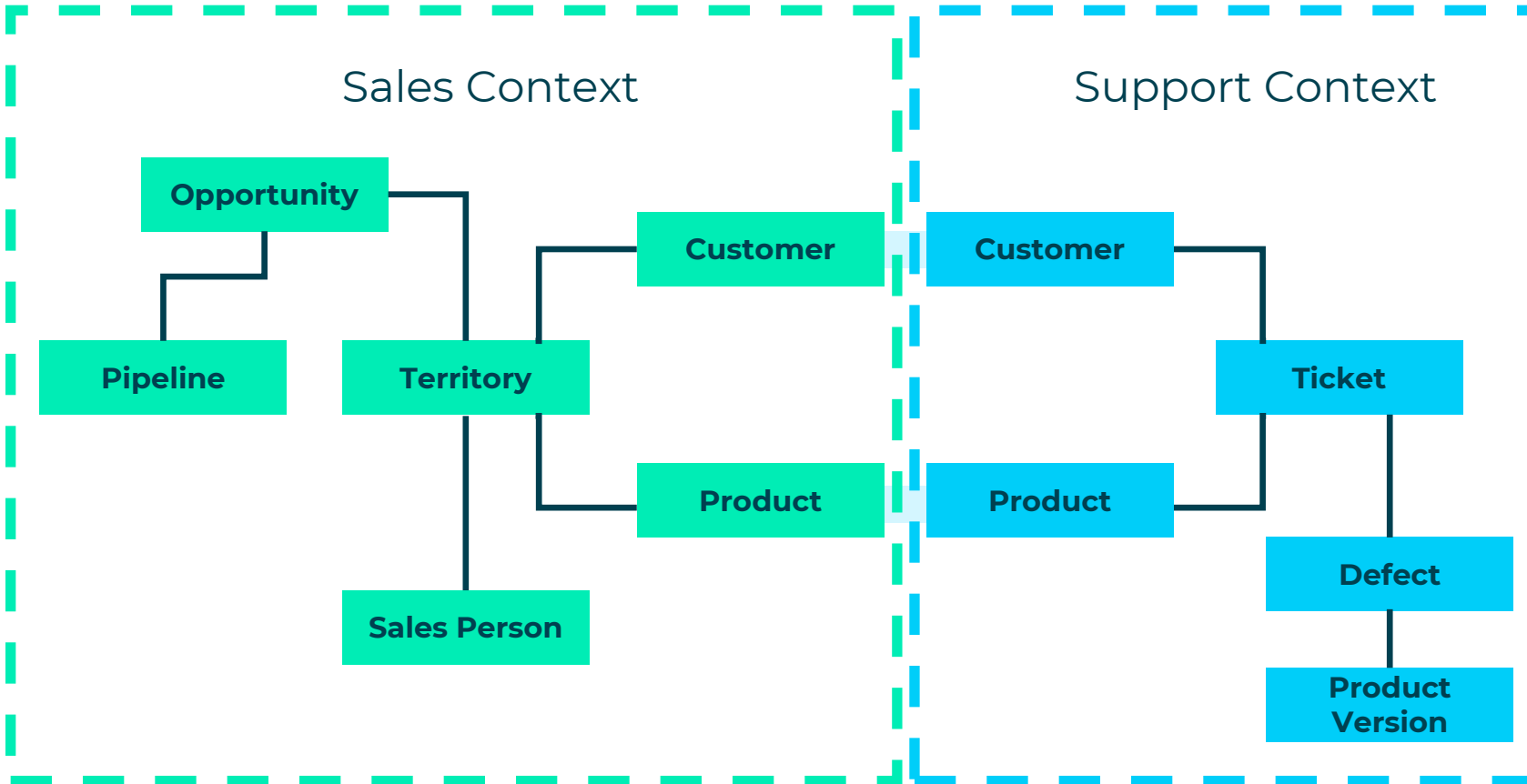


Domain Driven Design





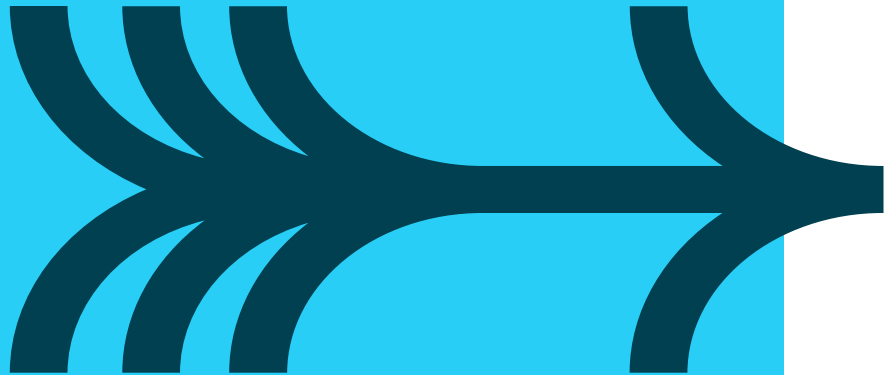
Bounded Context



- Quite often, the human factor is the dominant factor that draws boundaries between context – **Business Domains**
- They may have unrelated concepts (the support tickets)
- They may be shared concepts (customers and products)
- Interfaces must be maintained between these customers and products
 - *See note on API Contract and Semantic Versioning*



DOMAIN DRIVEN DESIGN & BOUNDED CONTEXT



Interface is a well-defined contract between domains

- Integration points between your domains, the API, is a **contract** that is maintained
- **Changes internally should not break the 'contract'** with other domains
- Follow Semantic versioning to unambiguously define a contract
 - **V1 > V1.1** etc backwards compatible API
 - **V1 > V2** etc no backwards compatibility maintained



MICROSERVICE DESIGN SUMMARY



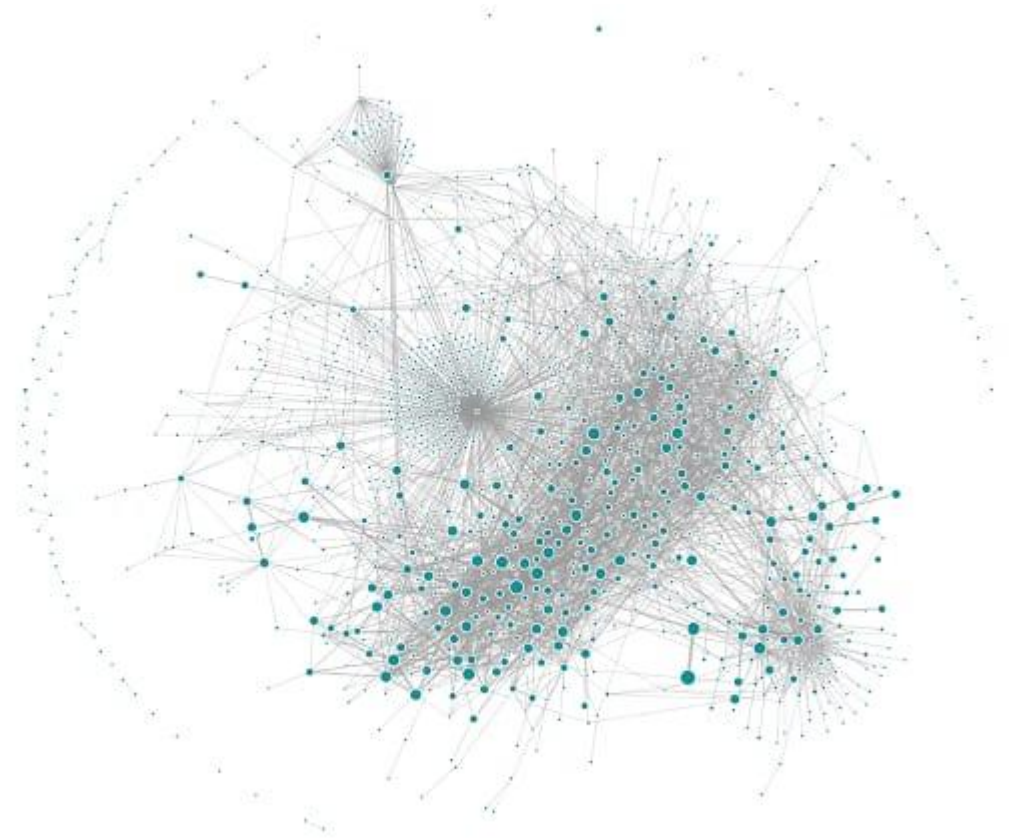
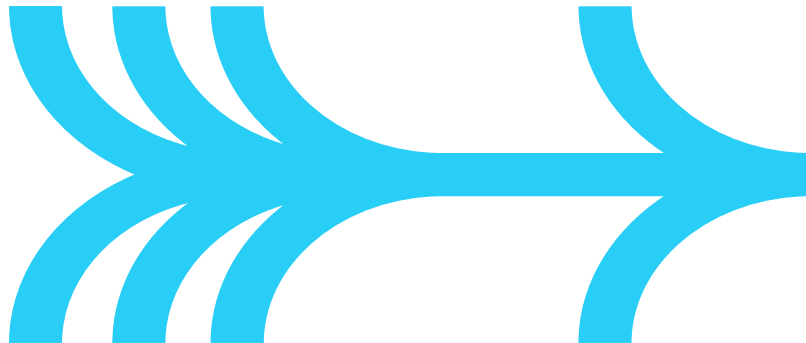
1. Focus on the Problem Domain
2. Ubiquitous Language
3. Bounded Contexts
4. Entities
Value Objects: (Customer and Address)
Aggregates
5. Repositories
6. Domain Events
7. Continuous Refinement

- **Order** (Aggregate Root - Entity):
 - **OrderLine** (Entity): Represents a customer's request
Specific product and quantity within an order.
 - **Product** (Entity): Is an entity the customer buys, referenced by OrderLine.
 - **Customer** (Entity): Customer information, such as name and address.
 - **Address** (Value Object)



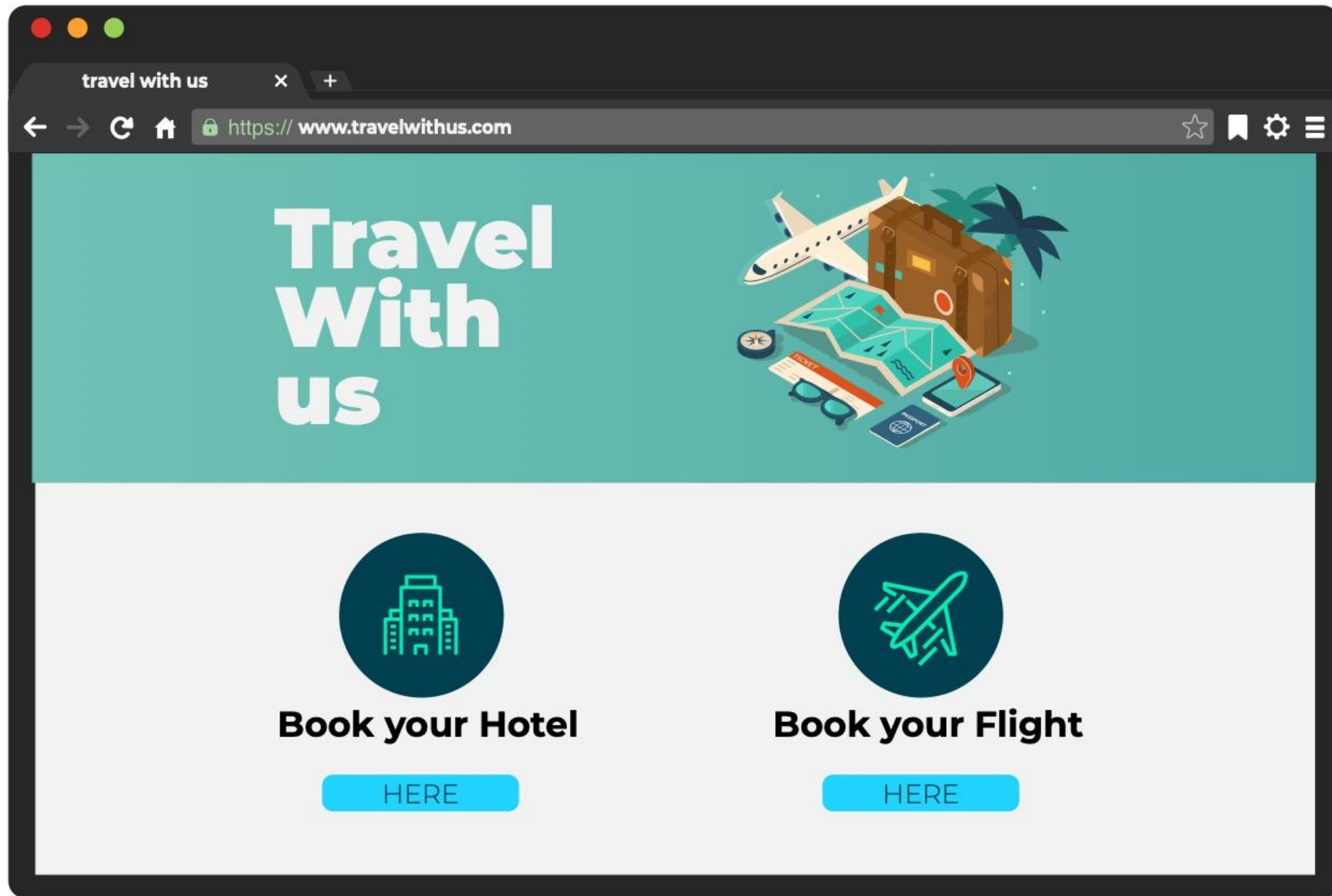
Challenges

1. Distributed system complexities
2. Increased operational overhead.
3. Proper planning, monitoring are essential.
4. Continuous improvement and feedback loops
Refine and optimize the architecture over time.



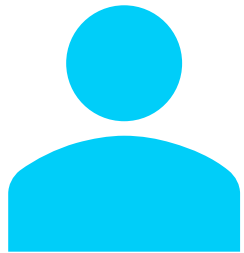
Uber's microservice architecture.

<https://www.uber.com/en-NL/blog/microservice-architecture/>

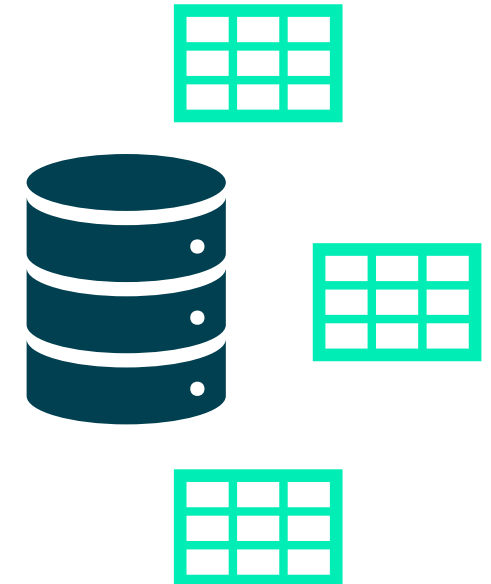
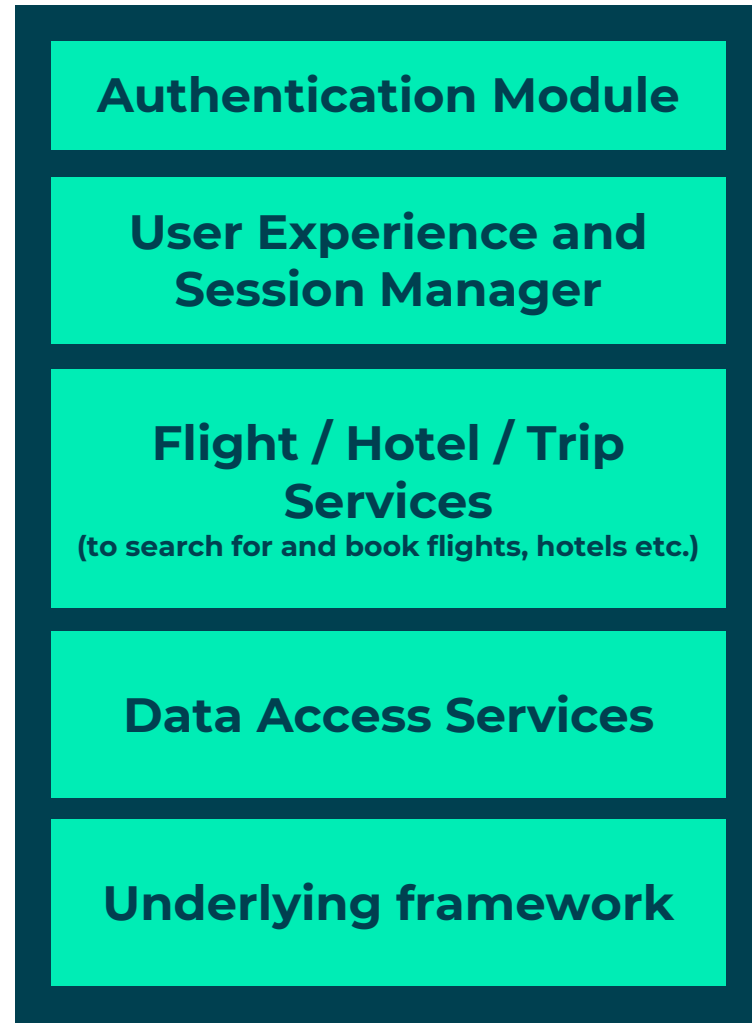




Internet



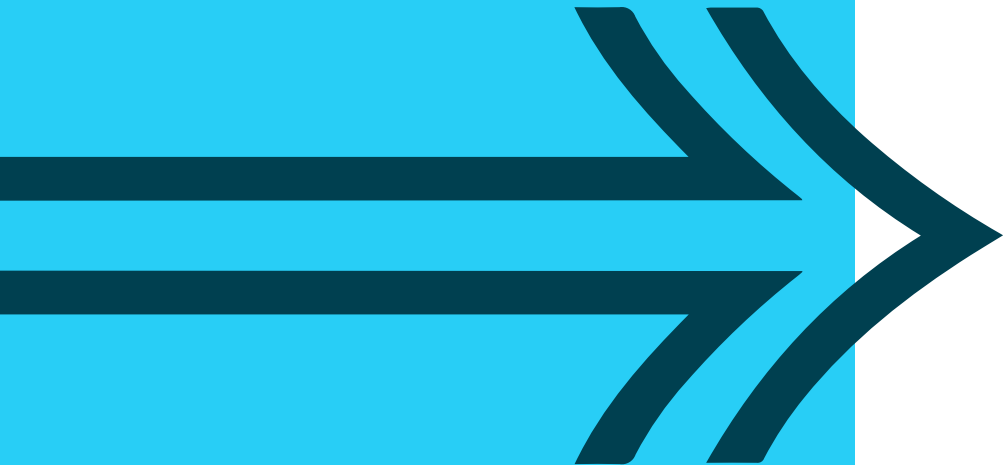
User



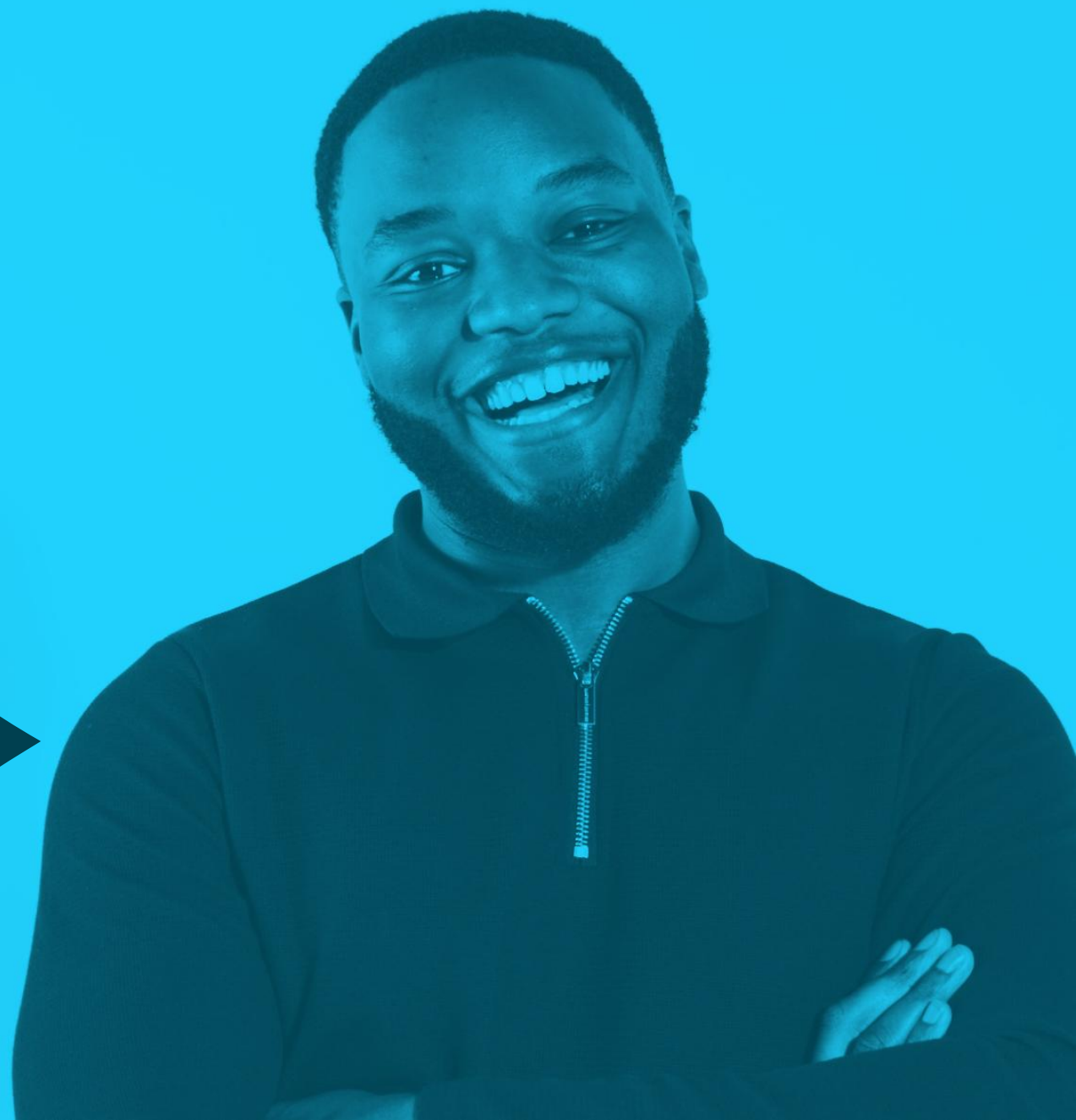
Monolithic Database
(many Tables)



GROUP ACTIVITY BREAKING THE MONOLITH



- Split into small groups
- Each group will look at the sample travel store Monolith in the previous slide and using the principals of Domain Driven Design & Bounded Context, will break it into Microservices.
- Draw a diagram of your design
- Present to the class the decisions made:
- Include the challenges the business or developer will face as they break down the monolith
- Make assumptions where appropriate but document and explain them
- How granular should the Microservice be? Why?
- How you may strangle the monolith as you go on the journey from Monolith to Microservice





- **Product Catalogue Service:**

- Manages product information, including details like name, description, price, and images. This service can handle search functionality, categorization, and filtering.

- **Inventory Service:**

- Tracks the availability of products in the inventory. It can handle functions such as updating stock levels, managing product reservations, and notifying when stock is low.

- **Order Service:**

- Manages the order lifecycle, from order creation to fulfilment. It can include features like order validation, order tracking, and order history.

- **User Service:**

- Handles user authentication, authorization, and user profile management. This service ensures that users can securely log in, manage their accounts, and track their orders.

- **Payment Service:**

- Deals with payment processing, including payment gateway integration, handling transactions, and managing payment-related data.



- **Cart Service:**
 - Shopping carts for users, add, update, and remove items.
- **Shipping Service:**
 - Shipping carriers to calculate shipping costs, generate shipping labels, and tracking.
- **Review and Rating Service:**
 - Product reviews and ratings. Feedback, trustworthy reputation.
- **Notification Service:**
 - User notification, order confirmation, shipping updates, promotional messages.
- **Recommendation Service:**
 - Product recommendations based on user behaviour, preferences, and purchase history.
- **Gateway Service:**
 - Entry point for client requests, Routing to microservices also authentication and authorisation.



- **Monitoring and Logging:**
 - Implement monitoring and logging within each microservice to track performance, errors, and overall system health.
 -
- **Database per Microservice**
 - Consider using database per microservice to maintain independence and data isolation.
- **Communication Between Microservices:**
 - RESTful APIs, Message queues for inter-service communication. API gateways for external communication.
- **Containerization and Orchestration:**
 - Like Docker, orchestrate them with tools like Kubernetes for deployment, scaling, and management.
 - Scalability: Independently scalable. Container orchestration to scale specific services based on demand.
- **Security:** HTTPS, authentication and authorization, regular security audits.
- **Testing:** Unit testing, integration testing, and end-to-end testing.